

LAPORAN PRAKTIKUM 12
DOUBLE LINKED LIST



NAMA : Fadila Mayamasita

NIM : 254107020012

KELAS : 1C TI

SARJANA TERAPAN TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG 2026

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List 12.2.1 Langkah-Langkah Percobaan

Pada percobaan 1 ini akan dibuat class Mahasiswa, Node, dan DoubleLinkedList. Class tersebut digunakan untuk mengimplementasikan struktur data Double Linked List beserta beberapa operasi dasar seperti:

- penambahan data di awal linked list,
- penambahan data di akhir linked list,
- pencarian data,
- penyisipan node setelah data tertentu,
- dan menampilkan isi linked list.

Langkah-langkah percobaan untuk menerapkan beberapa operasi dasar pada Double Linked List adalah sebagai berikut:

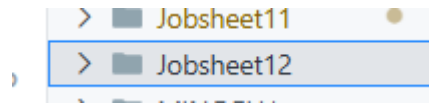
1. Perhatikan diagram class Mahasiswa, Node dan class DoubleLinkedList berikut. Diagram class berikut digunakan sebagai acuan dalam pembuatan program Double Linked List. Perhatikan hubungan antar class dan atribut yang digunakan pada setiap node.'

Mahasiswa<noPresensi>
nim: String nama: String kelas: String ipk: Double
Mahasiswa(String nim, String nama, String kelas, Double IPK) tampil()

Node<noPresensi>
data: Mahasiswa prev: Node next: Node
Node(prev:null, data: Mahasiswa data, next: null)

DoubleLinkedList<noPresensi>
head: Node tail : Node
DoubleLinkedLists() isEmpty(): boolean addFirst(): void addLast(): void add(data: Mahasiswa, index: int): void print(): void removeFirst(): void removeLast(): void search(): null insertAfter: void

2. Pada Project yang sudah dibuat pada Minggu sebelumnya, buat folder atau package baru bernama Jobsheet12 di dalam repository Praktikum ASD.



Catatan: Perhatikan setiap perubahan pointer prev dan next pada setiap operasi linked list. Kesalahan perubahan pointer dapat menyebabkan node terputus atau linked list tidak dapat ditelusuri dengan benar.

3. Buat class di dalam paket tersebut dengan nama Mahasiswa. Di dalam class tersebut, deklarasikan atribut sesuai dengan diagram class di atas. Tambahkan konstruktor untuk menginisialisasi seluruh atribut serta method tampil() untuk menampilkan data mahasiswa.

```
Sheet12 / I:\id\idb\idb\1.3\java / java / I:\id\idb\idb\1.3
1 package Jobsheet12;
2
3 //method untuk menyimpan data mahasiswa
4 public class Mahasiswa13 {
5     String nim;
6     String nama;
7     String kelas;
8     double ipk;
9
10    // constructor untuk menginisialisasi data mahasiswa
11    public Mahasiswa13(String nim, String nama, String kelas, double ipk) {
12        this.nim = nim;
13        this.nama = nama;
14        this.kelas = kelas;
15        this.ipk = ipk;
16    }
17    // method untuk menampilkan data mahasiswa
18    public void tampil() {
19        System.out.println(
20            "NIM      : " + nim +
21            "\nNama    : " + nama +
22            "\nKelas  : " + kelas +
23            "\nIPK     : " + ipk
24        );
25    }
26 }
```

4. Buat class di dalam paket tersebut dengan nama Node. Di dalam class tersebut, deklarasikan atribut data, prev, dan next sesuai diagram class. Selanjutnya tambahkan konstruktor sesuai class diagram.

```
Sheet12 / I:\id\idb\idb\1.3\java / java / I:\id\idb\idb\1.3
package Jobsheet12;

//method untuk menyimpan data mahasiswa
public class Node13 {
    Mahasiswa13 data;
    Node13 prev;
    Node13 next;

    // constructor untuk menginisialisasi data mahasiswa
    public Node13(Mahasiswa13 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

5. Buatlah sebuah class baru bernama DoubleLinkedList pada package yang sama dengan Node. Pada class DoubleLinkedList tersebut, deklarasikan atribut head dan tail pada class DoubleLinkedList.

```
//method double linked list untuk menyimpan data mahasiswa
public class DoubleLinkedList13 {
    Node13 head;
    Node13 tail;
}
```

6. Selanjutnya, buat konstruktor pada class DoubleLinkedList.

```
// constructor untuk menginisialisasi double linked list
public DoubleLinkedList13() {
    this.head = null;
    this.tail = null;
}
```

7. Buat method isEmpty(). Method isEmpty() digunakan untuk mengecek apakah linked list masih kosong atau sudah memiliki node. Method ini akan bernilai true jika head masih bernilai null.

```
// isEmpty untuk mengecek apakah double linked list kosong atau tidak
public boolean isEmpty() {
    return head == null;
}
```

8. Kemudian, buat method addFirst(). Method addFirst() digunakan untuk menambahkan node baru pada bagian awal linked list. Perhatikan perubahan pointer head dan prev setelah node baru ditambahkan.

```
// addFirst untuk menambahkan data mahasiswa di awal double linked list
public void addFirst(Mahasiswa13 data) {
    Node13 newNode = new Node13(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}
```

9. Selain itu pembuatan method `addLast()` digunakan untuk menambahkan node baru pada bagian akhir linked list. Perhatikan perubahan pointer tail dan next setelah proses penambahan dilakukan.

```
//addLast untuk menambahkan data mahasiswa di akhir double linked list
public void addLast(Mahasiswa13 data) {
    Node13 newNode = new Node13(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}
```

10. Method `insertAfter()` digunakan untuk menyisipkan node baru setelah node yang memiliki NIM tertentu. Perhatikan perubahan pointer next dan prev pada proses penyisipan node. Urutan perubahan pointer harus benar agar hubungan antar node tidak terputus.

```
//InsertAfter untuk menambahkan data mahasiswa setelah data tertentu di double linked list
public void insertAfter(String keyNim, Mahasiswa13 data) {
    Node13 current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }
    if (current == null) {
        System.out.println( "Data dengan NIM" + keyNim + " tidak ditemukan.");
        return;
    }

    Node13 newNode = new Node13(data);

    // jika current adalah tail, node baru ditambahkan di akhir
    if(current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else { // node baru disipkan di tengah
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}
```

11. Untuk mencetak isi dari linked lists dibuat method print(). Method print() digunakan untuk melakukan traversal dari node head hingga tail dan menampilkan seluruh isi linked list.

```
// print untuk menampilkan data mahasiswa di double linked list
public void print() {
    if (isEmpty()) {
        System.out.println(x: "Linked list masih kosong");
        return;
    }

    Node13 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}
```

12. Selanjutnya buat class DoubleLinkedListMain untuk mengeksekusi semua method yang ada pada class DoubleLinkedList.

```
-----

// import untuk menggunakan kelas Scanner
import java.util.Scanner;

// method untuk menguji kelas double linked list
public class DoubleLinkedListMain13 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList13 list = new DoubleLinkedList13();
        int pilihan;
    }
}
```

13. Buatlah menu pilihan pada class DoubleLinkedListMain. Gunakan struktur perulangan do-while agar program terus berjalan sampai pengguna memilih menu keluar.

```
do {
    System.out.println(x: "\n===== MENU DOUBLE LINKED LIST =====");
    System.out.println(x: "1. Tambah data di awal");
    System.out.println(x: "2. Tambah data di akhir");
    System.out.println(x: "3. Sisipkan data di tengah (setelah NIM)");
    System.out.println(x: "4. Hapus data awal");
    System.out.println(x: "5. Hapus data akhir");
    System.out.println(x: "6. Tampilkan data");
    System.out.println(x: "0. Keluar");
    System.out.print(s: "Pilih menu : ");
    pilihan = scan.nextInt();
    scan.nextLine();
}
```

14. Tambahkan switch-case untuk menjalankan menu pilihan program.

```
switch(pilihan) {  
    case 1:  
        Mahasiswa13 mhsAwal = inputMahasiswa(scan);  
        list.addFirst(mhsAwal);  
        break;  
    case 2:  
        Mahasiswa13 mhsAkhir = inputMahasiswa(scan);  
        list.addLast(mhsAkhir);  
        break;  
    case 3:  
        System.out.print(s: "Masukkan NIM yang dicari : ");  
        String keyNim = scan.nextLine();  
        System.out.println(x: "Masukkan data baru: ");  
        Mahasiswa13 dataBaru = inputMahasiswa(scan);  
        break;  
    case 4:  
        list.removeFirst();  
        break;  
    case 5:  
        list.removeLast();  
        break;  
    case 6:  
        list.print();  
        break;  
    case 0:  
        System.out.println(x: "Program selesai.");  
        break;  
    default:  
        System.out.println(x: "Menu tidak valid");  
}  
} while (true);
```

15. Untuk menerima input data mahasiswa yang akan disimpan ke dalam list, buat method `inputMahasiswa()` di dalam class `DoubleLinkedListMain`.

```
public static Mahasiswa13 inputMahasiswa(Scanner scan) {  
    System.out.print(s: "Masukkan NIM    : ");  
    String nim = scan.nextLine();  
  
    System.out.print(s: "Masukkan Nama  : ");  
    String nama = scan.nextLine();  
  
    System.out.print(s: "Masukkan Kelas : ");  
    String kelas = scan.nextLine();  
  
    System.out.print(s: "Masukkan IPK   : ");  
    double ipk = scan.nextDouble();  
    scan.nextLine();  
  
    // Memanggil konstruktor class Mahasiswa13  
    return new Mahasiswa13(nim, nama, kelas, ipk);  
}
```

12.2.2 Verifikasi Hasil Percobaan

```
===== MENU DOUBLE LINKED LIST =====  
1. Tambah data di awal  
2. Tambah data di akhir  
3. Sisipkan data di tengah (setelah NIM  
4. Hapus data awal  
5. Hapus data akhir  
6. Tampilkan data  
0. Keluar  
Pilih menu : 2  
Masukkan NIM    : 123005  
Masukkan Nama   : Harry  
Masukkan Kelas  : 1A  
Masukkan IPK    : 3,76
```

```
===== MENU DOUBLE LINKED LIST =====  
1. Tambah data di awal  
2. Tambah data di akhir  
3. Sisipkan data di tengah (setelah NIM  
4. Hapus data awal  
5. Hapus data akhir  
6. Tampilkan data  
0. Keluar  
Pilih menu : 3  
Masukkan NIM yang dicari: 123005  
Masukkan data baru:  
Masukkan NIM    : 123010  
Masukkan Nama   : Potter  
Masukkan Kelas  : 1B  
Masukkan IPK    : 3,55  
Data berhasil disisipkan setelah NIM 123005
```

```
===== MENU DOUBLE LINKED LIST =====  
1. Tambah data di awal  
2. Tambah data di akhir  
3. Sisipkan data di tengah (setelah NIM  
4. Hapus data awal  
5. Hapus data akhir  
6. Tampilkan data  
0. Keluar  
Pilih menu : 6  
NIM      : 123005  
Nama     : Harry  
Kelas   : 1A  
IPK      : 3.76  
NIM      : 123010  
Nama     : Potter  
Kelas   : 1B  
IPK      : 3.55
```


12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawaban :

Struktur Single Linked List setiap nodenya hanya mempunyai satu buah pointer penghubung yaitu next. Sedangkan Double Linked List setiap nodenya hanya mempunyai dua buah pointer penghubung yaitu next dan prev

Mekanisme traversal pada SLL, penelusurannya hanya bisa dilakukan searah atau maju saja (head → depan), sedangkan di DLL penelusuran dapat dilakukan secara dua arah, baik bergerak maju ke head ke tail maupun bergerak mundur dari tail ke head.

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawaban :

Atribut next: Berfungsi sebagai pointer yang menyimpan alamat memori dari node selanjutnya di dalam list. Digunakan untuk navigasi/traversal maju

Atribut prev: Berfungsi sebagai pointer yang menyimpan alamat memori dari node sebelumnya di dalam list. Digunakan untuk navigasi mundur serta mempermudah manipulasi pointer (seperti penyisipan atau penghapusan data di tengah) tanpa harus mencari node pendahulu dari awal list.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawaban :

Untuk menginisialisasi atau menciptakan objek linked list baru dalam keadaan kosong. Dengan memberikan nilai null pada head dan tail, program mengetahui bahwa belum ada satu pun node atau data yang tersimpan di dalam list tersebut.

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawaban :

Karena ketika sebuah node baru ditambahkan ke dalam linked list yang semula kosong, node tersebut otomatis bertindak sebagai node pertama sekaligus node terakhir di dalam list tersebut. Oleh karena itu, pointer head (penunjuk awal) dan tail (penunjuk akhir) wajib mengarah ke satu node tunggal yang sama.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

Jawaban :

```
// print untuk menampilkan data mahasiswa di double linked list  
public void print() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked list masih kosong");  
        return;  
    }  
    Node13 current = head;  
    while (current != null) {  
        current.data.tampil();  
        current = current.next;  
    }  
}
```

6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

```
// method untuk menampilkan data secara terbalik dari tail ke head  
public void printReverse() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked list masih koasong");  
        return;  
    }  
  
    Node13 current = tail;  
  
    while (current != null) {  
        current.data.tampil();  
        current = current.prev;  
    }  
}
```

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

12.3.1 Langkah-Langkah Percobaan

Pada percobaan 2 akan dibuat beberapa method untuk menghapus node pada Double Linked List. Penghapusan dilakukan melalui:

- node paling depan (removeFirst)
- node paling belakang (removeLast)

1. Buatlah method removeFirst() di dalam class DoubleLinkedList untuk menghapus node pada bagian awal linked list. Perhatikan perubahan pointer head setelah node pertama dihapus.

```
// method untuk menghapus data mahasiswa di awal double linked list
public void removeFirst() {
    if(isEmpty()) {
        System.out.println(x: "Linked list kosong..");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}
```

2. Tambahkan method removeLast() untuk menghapus node pada bagian akhir linked list. Perhatikan perubahan pointer tail setelah proses penghapusan dilakukan.

```
// method untuk menghapus data mahasiswa di akhir double linked list
public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked list kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
```

12.3.2 Verifikasi Hasil Percobaan

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data awal
5. Hapus data akhir
6. Tampilkan data
0. Keluar
Pilih menu : 5
Data berhasil dihapus.
NIM      : 123010
Nama     : Potter
Kelas   : 1B
IPK      : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data awal
5. Hapus data akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76
```

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```
head = head.next;
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawaban :

`head = head.next;` berfungsi untuk memindahkan posisi pointer utama `head` agar bergeser ke node kedua. Dengan begitu, posisi node pertama secara logis terlepas dari struktur list.

head.prev = null; berfungsi untuk memutus hubungan/pointer mundur dari node kedua tersebut menuju node pertama lama yang ingin dihapus. Karena node kedua kini telah resmi menjadi kepala list yang baru, ia tidak boleh memiliki penunjuk ke node sebelumnya (null).

2. Modifikasi method removeFirst() dan removeLast() agar program menampilkan data yang berhasil dihapus!

```
// method untuk menghapus data mahasiswa di awal double linked list
public void removeFirst() {
    if(isEmpty()) {
        System.out.println(x: "Linked list kosong..");
        return;
    }
    System.out.println(x: "Data berhasil dihapus");
    head.data.tampil();
    System.out.println();

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}

// method untuk menghapus data mahasiswa di akhir double linked list
public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked list kosong.");
        return;
    }

    System.out.println(x: "Data berhasil dihapus.");
    tail.data.tampil();
    System.out.println();

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
```